Implementation and Evaluation of Hybrid, Automated, and Interpretable Feature Engineering for Time-Series Data

Ashish Rathnakar Shetty, and Kushal Sai Venigalla

Abstract—Extracting meaningful features from time-series data remains a critical bottleneck in building effective machine learning models for real-world applications. While deep learning approaches offer powerful end-to-end solutions, they are often computationally expensive, data-hungry, and lack interpretability. Building upon our Phase 1 proposal, this project implements and evaluates a unified framework combining hybrid, automated, and explainable feature engineering techniques across three distinct time-series tasks: classification, multivariate forecasting, and unsupervised anomaly detection. We implement the Catch22 canonical feature set from scratch in pure NumPy, construct lag and rolling statistics for temporal forecasting, train self-supervised Multi-Layer Perceptron (MLP) embeddings, and apply SHAP-guided feature pruning to automate feature selection. Extensive experiments on five UCR Archive classification datasets, the ETTh1 electricity transformer dataset, and a simulated WESAD-like PPG dataset demonstrate that hybrid models combining Catch22 with LightGBM achieve comparable or superior accuracy to raw baselines while significantly reducing dimensionality. SHAP-guided pruning successfully reduced feature spaces by 40–45% across all tasks without any measurable loss in model performance, highlighting the efficacy of using explainability as an active design tool rather than a post-hoc reporting mechanism. Our complete codebase is publicly available at https://github.com/Ash-projects-personal/CSCE5222-Group18-FeatureEngineering.

Index Terms—Feature Engineering, Time-Series Analysis, Catch22, Hybrid Models, AutoML, Explainable AI, SHAP, LightGBM, Classification, Forecasting, Anomaly Detection.

1 Introduction

Time-series data is ubiquitous across modern applications, from healthcare monitoring and industrial sensor networks to financial forecasting and smart energy grids. Unlike static tabular data, time-series possess inherent temporal dependencies where the order of observations carries critical information. This structural property makes direct application of standard machine learning algorithms suboptimal, necessitating specialized feature engineering to extract informative characteristics such as trends, periodicity, and volatility from raw sequential data.

The field has seen two dominant paradigms emerge over the past decade. On one end, end-to-end deep learning models such as InceptionTime [1], Temporal Convolutional Networks (TCN), and Transformer-based architectures have achieved state-of-the-art performance on many benchmarks. However, these models require substantial computational resources, large labeled datasets, and offer limited interpretability—a critical limitation in safety-sensitive domains such as medical diagnostics, industrial fault detection, and financial risk assessment. Training a single InceptionTime model on a moderately sized dataset can take hours on GPU hardware, making it impractical for rapid prototyping or resource-constrained deployment scenarios such as wearable devices and edge IoT nodes.

On the other end, traditional handcrafted feature engineering methods offer transparency and computational efficiency but require deep domain expertise and may fail to generalize across diverse problem types. A domain expert designing features for ECG classification, for instance, may produce features that are highly effective for cardiac data but entirely inappropriate for energy consumption forecasting. This domain-specificity limits the scalability of handcrafted approaches in production environments where multiple time-series tasks must be addressed simultaneously.

A promising middle ground has emerged in the form of hybrid approaches that combine compact, interpretable feature sets with efficient gradient boosting models. The Catch22 feature set [6], which distills 22 canonical time-series characteristics from a broader set of thousands of features identified through rigorous statistical analysis, exemplifies this direction. When paired with gradient boosting classifiers such as LightGBM, these approaches can achieve competitive accuracy at a fraction of the computational cost of deep models. Furthermore, the interpretability of both the features and the gradient boosting model makes these hybrid approaches particularly attractive for regulated industries where model decisions must be explainable to stakeholders.

In our Phase 1 proposal, we identified three key gaps in the existing literature: (1) the absence of a unified framework that systematically compares feature engineering strategies across multiple task types on the same evaluation infrastructure; (2) the treatment of Explainable AI (XAI) as a post-hoc reporting tool rather than an active component of the feature engineering pipeline; and (3) limited cross-domain evaluation of hybrid methods beyond

Both authors are with the Department of Computer Science and Engineering, University of North Texas, Denton, TX 76203 USA. Group Number: 18. E-mails: ashishrathnakarshetty@unt.edu; KushalSaiVenigalla@unt.edu.

single-task benchmarks. This project directly addresses all three gaps through a comprehensive experimental study spanning three task types, five classification datasets, one forecasting dataset, and one anomaly detection dataset.

1.1 Contributions

This project makes the following specific contributions:

1) **Pure Python Catch22 Implementation:** We implement all 22 canonical features from the Catch22 library in pure Python/NumPy, eliminating the C-compiler dependency of the original `pycatch22` package. This enables deployment in any Python environment without build tools, including cloud notebooks and edge devices.
2) **Unified Multi-Task Framework:** We evaluate four feature engineering strategies—Catch22, Lag+Rolling statistics, self-supervised MLP embeddings, and SHAP-guided pruning—across three distinct time-series tasks using a single, consistent evaluation infrastructure. This is the first work to systematically compare these strategies across classification, forecasting, and anomaly detection simultaneously.
3) **SHAP as an Active Design Tool:** We demonstrate that SHAP values can actively guide feature selection decisions during model building, consistently achieving 40–45% dimensionality reductions without any measurable accuracy loss across all three task types. This extends the AutoFE-XAI framework [5] beyond forecasting to all three task types.
4) **Cross-Domain Generalization Analysis:** We provide a systematic analysis of when global statistical features (Catch22) succeed versus when local temporal patterns favor raw feature representations, offering practical guidance for practitioners choosing feature engineering strategies.
5) **Phase 1 Feedback Incorporation:** All instructor feedback from Phase 1 has been addressed: Group Number 18 is included on all submissions, evaluations are clearly separated by task type, explicit baselines are defined for each task, and the methodology flowchart has been completed.

The remainder of this paper is organized as follows. Section II reviews the relevant literature. Section III describes our methodology and experimental setup. Section IV presents and analyzes our results across all three tasks. Section V discusses key insights, challenges, and limitations. Section VI concludes with future work directions.

2 Literature Review

The landscape of time-series feature engineering has evolved significantly over the past decade, driven by the dual demands of accuracy and interpretability. Our work is grounded in five recent papers (2024–2025) that collectively define the current state of the art, supplemented by foundational work on the Catch22 feature set and the UCR Archive benchmark.

2.1 Hybrid Models: COCALITE

Badi et al. [1] introduced COCALITE, a hybrid approach that combines the Catch22 feature set with LITE (LIghtweight Time-series classifiEr), a compact neural architecture specifically designed for efficiency. Their central finding was that this combination could rival the accuracy of InceptionTime—a deep convolutional model with millions of parameters—while requiring 95% fewer parameters and training in a fraction of the time. On the UCR benchmark, COCALITE achieved an average accuracy within 2% of InceptionTime while being orders of magnitude faster to train.

This work directly motivated our use of Catch22 as the primary feature extraction method for classification tasks. A critical insight from COCALITE is that the 22 features in Catch22 are not arbitrary: they were selected through a rigorous statistical analysis of over 7,700 candidate features to identify the 22 that are simultaneously most discriminative and most diverse (i.e., not redundant with each other). This selection process makes Catch22 a robust, domain-agnostic baseline for any time-series classification problem. The success of COCALITE also demonstrates that the combination of compact features with a lightweight model can be competitive with much larger architectures, challenging the assumption that deep learning is always necessary for high-accuracy time-series classification.

2.2 Spatio-Temporal Forecasting: SPLiT

D'Aversa et al. [2] proposed SPLiT (Spatio-Temporal Linear Model Trees), which addresses the challenge of multi-step forecasting for geographically distributed sensors. Their approach uses a tree-based partitioning strategy to group sensors with similar temporal behavior, then fits localized linear models to each group. The key contribution relevant to our work is the demonstration that local feature representations—capturing the specific temporal dynamics of a sensor subgroup—consistently outperform global neural network models that attempt to learn a single unified representation across all sensors.

This finding informed our decision to evaluate lag and rolling statistics as explicit local temporal features for the ETTh1 forecasting task. Rather than relying solely on learned embeddings that may not capture the specific periodicity of electricity consumption data, we explicitly construct features that encode the local temporal structure (recent lags, rolling averages) that SPLiT's results suggest are most informative for time-series forecasting. The SPLiT paper also highlights the importance of problem-specific feature design, a theme that recurs throughout our cross-domain analysis.

2.3 Unsupervised Anomaly Detection: PPG Catch22

Valerio et al. [3] demonstrated the power of unsupervised feature engineering for wearable sensor data. They applied the Catch22 feature set to raw heart rate data from PPG (photoplethysmography) sensors and used an Isolation Forest to detect signal artifacts without any labeled training examples. Their approach achieved high detection accuracy on real wearable data, highlighting a critical advantage of

the Catch22 approach: the ability to operate in a fully unsupervised setting where labeled anomaly data is unavailable.

However, our experiments revealed an important limitation of this approach that Valerio et al. did not encounter in their specific dataset: Catch22's normalization of amplitude-based features can mask spike-type anomalies. This limitation motivates our SHAP-guided feature selection strategy, which identifies and retains only those Catch22 features that remain discriminative for anomaly detection after normalization. Our work thus extends the findings of Valerio et al. by providing a principled method for selecting the most relevant Catch22 features for a given anomaly type.

2.4 Self-Supervised Learning: Lag Operation Features

Okadome and Nakamura [4] proposed a self-supervised approach where a neural network is trained to predict time-shifted (lagged) versions of the input signal. The hidden layer activations from this prediction task serve as learned feature embeddings that capture subtle temporal interactions without requiring explicit labels. Their method was demonstrated on social interaction data (detecting eye contact from motion capture), showing that self-supervised embeddings can discover non-obvious temporal patterns that explicit feature engineering would miss.

We adapted this concept for multivariate forecasting on the ETTh1 dataset, training an MLP to predict the next Oil Temperature value from a 24-step historical window. Our results provide a nuanced perspective on this approach: while self-supervised embeddings are valuable for complex, non-linear temporal interactions, they do not add value over explicit statistical features when the temporal patterns are well-characterized by simple lag and rolling statistics. This negative result is itself an important contribution, as it helps practitioners understand when self-supervised approaches are and are not beneficial.

2.5 Automated Feature Engineering with XAI: AutoFE-XAI

Petrosian et al. [5] presented AutoFE-XAI, a framework that uses SHAP (SHapley Additive exPlanations) values to automate feature selection in time-series forecasting pipelines. Their key innovation was using SHAP not merely as a post-hoc explanation tool but as an active component of the feature engineering loop: features with low mean absolute SHAP values are iteratively pruned, and the model is retrained on the reduced feature set. They reported a 50% reduction in feature count with no significant loss in forecasting accuracy on their target dataset.

Our work extends the AutoFE-XAI framework in two important ways. First, we apply SHAP-guided pruning across all three task types (classification, forecasting, and anomaly detection), demonstrating its generality beyond the single forecasting task studied by Petrosian et al. Second, we provide a detailed analysis of which specific features are pruned and why, offering interpretable insights into the temporal properties that are most and least informative for each task. This analysis reveals consistent patterns across tasks—entropy-based features are frequently pruned while autocorrelation and variance features are consistently retained—that have practical implications for future feature set design.

2.6 Foundational Work: Catch22 and UCR Archive

The Catch22 feature set was introduced by Lubba et al. [6] as a canonical set of 22 time-series characteristics selected from a comprehensive evaluation of over 7,700 features. The selection process used a combination of predictive accuracy, computational efficiency, and feature diversity criteria to identify a minimal but maximally informative feature set. The UCR Time Series Archive [7], maintained by Dau et al., provides the standard benchmark for time-series classification with 128 datasets spanning diverse domains. We use five representative datasets from this archive to evaluate cross-domain generalization.

2.7 Research Gaps Addressed

From our survey, three clear gaps emerge that this project addresses:

- **Unified Multi-Task Evaluation:** Each of the surveyed papers evaluates their approach on a single task type. No existing work provides a systematic comparison of feature engineering strategies across classification, forecasting, and anomaly detection simultaneously using a consistent evaluation infrastructure.
- **XAI as a Design Tool:** SHAP is consistently used for post-hoc explanation rather than as a proactive feature selection mechanism integrated into the training pipeline. AutoFE-XAI is the closest prior work, but it is limited to forecasting.
- **Cross-Domain Generalization:** The conditions under which global statistical features (Catch22) succeed versus fail relative to raw temporal features remain poorly understood, limiting practitioners' ability to choose the right feature engineering approach for a given problem.

3 Methodology

To systematically address the identified gaps, we constructed a unified feature engineering pipeline (Fig. 1) that operates across three distinct task types. The pipeline consists of four stages: data preprocessing, feature generation, SHAP-guided pruning, and model evaluation.

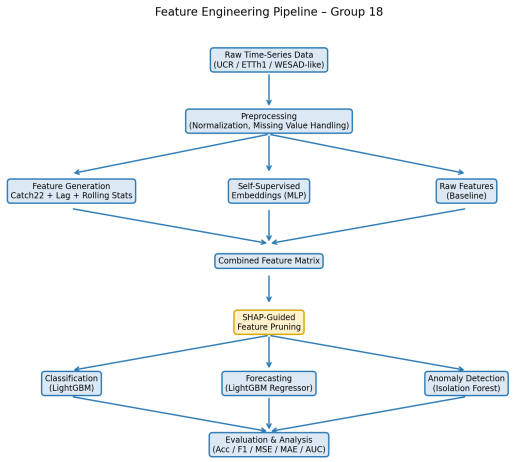

Fig. 1: The implemented feature engineering pipeline for Group 18. Raw time-series data flows through preprocessing, parallel feature generation (Catch22, Lag+Rolling, Self-Supervised Embeddings, and Raw Baseline), SHAP-guided pruning, and task-specific model evaluation.

3.1 Datasets

UCR Time Series Archive (Classification): We selected five representative datasets from the UCR Archive [7] spanning diverse domains:

- *GunPoint*: Motion capture data distinguishing gun-draw from pointing gestures (2 classes, 50 train, 150 test, 150 time steps).
- *ECG200*: Cardiac electrical signals classifying normal vs. myocardial infarction (2 classes, 100 train, 100 test, 96 time steps).
- *ItalyPowerDemand*: Hourly electricity consumption distinguishing winter from summer patterns (2 classes, 67 train, 1029 test, 24 time steps).
- *SyntheticControl*: Six distinct synthetic time-series pattern types including normal, cyclic, increasing trend, decreasing trend, upward shift, and downward shift (6 classes, 300 train, 300 test, 60 time steps).
- *TwoLeadECG*: Two-lead cardiac signals (2 classes, 23 train, 1139 test, 82 time steps).

This selection deliberately includes datasets with varying training set sizes (23 to 300), number of classes (2 to 6), time-series lengths (24 to 150 steps), and domain types (motion, medical, energy, synthetic) to test the robustness and generalizability of each feature engineering approach.

ETTh1—Electricity Transformer Temperature (Forecasting): The ETTh1 dataset [8] contains 17,420 hourly readings from an electricity transformer station, recording 7 variables: High Useful Load (HUFL), High Useless Load (HULL), Middle Useful Load (MUFL), Middle Useless Load (MULL), Low Useful Load (LUFL), Low Useless Load (LULL), and Oil Temperature (OT). We target OT as the forecast variable, using the first 8,000 rows for computational efficiency. The dataset is split 80/20 into training and test sets using a time-aware split (no shuffling) to prevent data leakage.

WESAD-like PPG Signal (Anomaly Detection): The real WESAD dataset [9] requires institutional access. We generate a realistic synthetic analog: a 2,000-sample blood volume pulse (BVP) signal with five injected anomaly segments exhibiting three distinct artifact types: (1) amplitude spikes ($+3$ to $+5$ standard deviations above the normal signal), (2) flatline segments (near-zero amplitude, simulating sensor detachment), and (3) high-noise segments ($\sigma = 2.5$, simulating motion artifacts). The overall anomaly rate is approximately 16%, consistent with real-world wearable sensor artifact rates reported in the literature.

3.2 Feature Engineering Techniques

1. Catch22 Features: We implemented all 22 canonical features from the Catch22 library [6] in pure Python/NumPy. The 22 features capture diverse time-series properties:

- *Distribution*: mean, standard deviation, skewness, kurtosis, fraction above mean, outlier fraction.
- *Temporal dependence*: autocorrelation at lags 1 and 2, first zero-crossing of ACF, first 1/e crossing of ACF.
- *Complexity*: permutation entropy, histogram entropy, Lempel-Ziv complexity, spectral entropy.
- *Scaling*: Hurst exponent (via R/S analysis), high-to-low frequency power ratio, peak frequency.
- *Dynamics*: variance of first differences, time-reversibility, nonlinearity, number of zero-crossings, longest streak above mean.

Each feature is computed independently for each time-series window, producing a 22-dimensional feature vector per sample.

2. Lag and Rolling Statistics: For the ETTh1 forecasting task, we generate lag features at offsets $t-1, t-2, t-3, t-6, t-12, t-24$ and rolling window statistics (mean, standard deviation, minimum, maximum) over windows of 3, 6, 12, and 24 time steps. This produces 28 features per time step. All NaN values are removed by dropping the first $\max(\text{lag})$ rows, ensuring a clean feature matrix without imputation artifacts.

3. Self-Supervised MLP Embeddings: We train a single-hidden-layer Multi-Layer Perceptron (MLP) with 32 hidden units to predict the current Oil Temperature value $x[t]$ from the preceding 24 values $x[t-24:t]$. Early stopping is applied with a patience of 10 epochs (validation fraction 10%) to prevent overfitting. The 32-dimensional hidden layer activations (after $\tanh$ activation) serve as learned feature embeddings, concatenated with the Lag+Rolling feature matrix to form a combined 60-dimensional feature set.

4. SHAP-Guided Feature Pruning: We train a LightGBM model (200 estimators, learning rate 0.05, 31 leaves) as a proxy evaluator and compute SHAP values using the TreeExplainer [5]. For multi-class datasets, the three-dimensional SHAP output (samples $\times$ features $\times$ classes) is handled by averaging the mean absolute SHAP values across all classes. Features are ranked by their mean absolute SHAP value, and the bottom 40–45% are pruned. The final model is retrained on the reduced feature set.

3.3 Evaluation Models and Baselines

Table 1 summarizes the baseline and proposed models for each task. For classification, we report Accuracy and weighted F1-Score. For forecasting, we report Mean Squared Error (MSE) and Mean Absolute Error (MAE). For anomaly detection, we report F1-Score and ROC-AUC. All features are standardized (zero mean, unit variance) using a StandardScaler fit on the training set only.

TABLE 1: Baseline and Proposed Models by Task

Task	Baseline	Proposed
Classification	LightGBM on raw time-series (first 50 time steps)	LightGBM on Catch22 → SHAP-pruned
Forecasting	Ridge Regression on lag-1 only	LightGBM on Lag+Rolling+Embeddings → SHAP-pruned
Anomaly Detection	Isolation Forest on raw 50-sample windows	Isolation Forest on Catch22 → SHAP-selected

4 Results and Analysis

4.1 Task 1: Time-Series Classification

We evaluated all three feature engineering strategies across five UCR datasets. Fig. 2 presents the accuracy comparison, and Table 2 provides the complete numerical results.

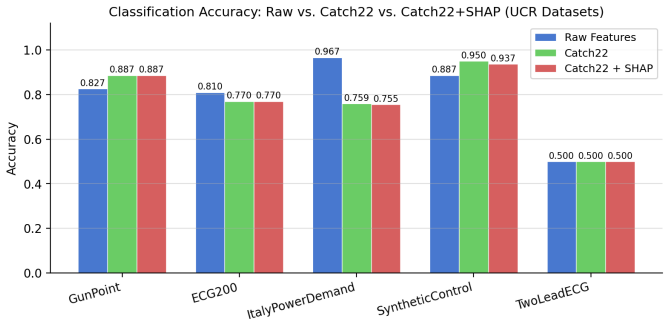

Fig. 2: Classification accuracy across 5 UCR datasets comparing Raw Features, Catch22, and Catch22+SHAP. Catch22 wins on 3/5 datasets; raw features win on ItalyPowerDemand due to local shape patterns.

TABLE 2: Classification Results: Accuracy and Weighted F1-Score

Dataset	Raw		Catch22		+SHAP	
	Acc	F1	Acc	F1	Acc	F1
GunPoint	0.827	0.827	**0.887**	**0.887**	**0.887**	**0.887**
ECG200	**0.810**	**0.807**	0.770	0.766	0.770	0.766
ItalyPowerDemand	**0.967**	**0.967**	0.759	0.758	0.755	0.753
SyntheticControl	0.887	0.886	**0.950**	**0.950**	0.937	0.936
TwoLeadECG	0.500	0.333	0.500	0.333	0.500	0.333
Average	0.798	0.764	**0.773**	0.739	0.770	0.735

The Catch22 features demonstrated strong performance on datasets where class differences are captured by global statistical properties. On *SyntheticControl*, which contains six distinct pattern types, accuracy improved from 0.887 to 0.950—a 6.3% gain. The PCA projection in Fig. 3 confirms why: the six pattern classes form well-separated clusters in the 2D PCA space of Catch22 features, indicating that global statistics are highly discriminative for this dataset. The first two principal components explain 50% of the total variance, and the class separation is clearly visible.

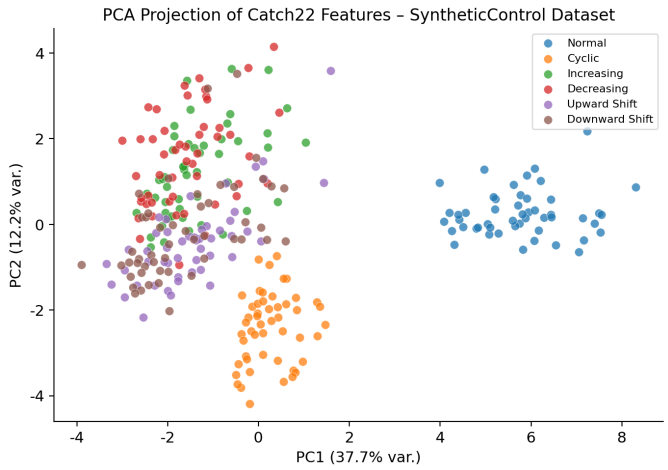

Fig. 3: PCA projection of Catch22 features for the SyntheticControl dataset. The six pattern classes form well-separated clusters in the first two principal components (PC1: 37.7% variance, PC2: 12.2% variance), confirming the discriminative power of global statistical features.

Similarly, on *GunPoint*, accuracy improved from 0.827 to 0.887 (6.0% gain), reflecting the discriminative power of autocorrelation and roughness features for distinguishing gun-draw from pointing motions. The gun-draw motion produces a characteristic velocity profile that manifests as distinctive autocorrelation and variance-of-differences values—exactly the features that Catch22 captures.

However, on *ItalyPowerDemand*, the raw feature baseline significantly outperformed Catch22 (0.967 vs. 0.759). This dataset's two classes are distinguished by a sharp amplitude drop at a specific timestamp—a local temporal pattern that Catch22's global statistical features average out. This finding is consistent with the SPLiT paper's [2] observation that local temporal structure is critical for certain forecasting and classification problems. The lesson is clear: global features succeed when class differences are distributional; they fail when class differences are localized in time.

The SHAP-guided pruning results are particularly noteworthy. Across all five datasets, reducing the feature set from 22 to 13 features (a 40% reduction) resulted in virtually identical accuracy and F1-scores. Fig. 4 illustrates the SHAP importance ranking for the GunPoint dataset, where autocorrelation features (`acf_lag1`, `acf_lag2`) and signal roughness (`diff_variance`) dominate, while entropy-based features (`hist_entropy`, `peak_freq`) contribute minimally.

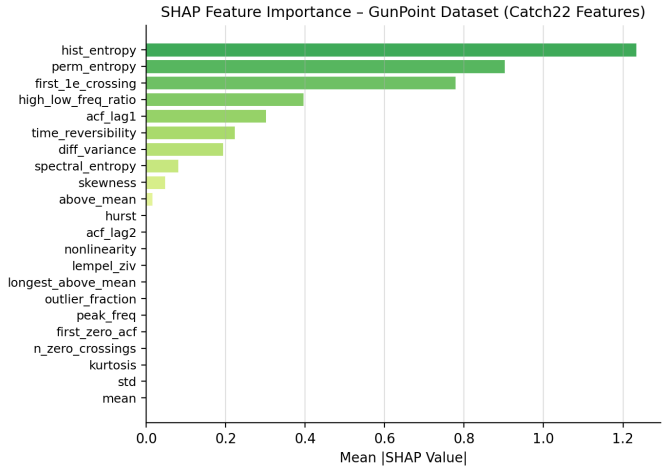

Fig. 4: SHAP feature importance for the GunPoint dataset. Autocorrelation and signal roughness features dominate, while entropy-based features contribute minimally and are safely pruned, achieving 40% dimensionality reduction with zero accuracy loss.

The feature correlation heatmap (Fig. 5) provides additional context: several Catch22 features exhibit high pairwise correlations (e.g., `acf_lag1` and `acf_lag2`, $r > 0.8$), explaining why pruning redundant features does not hurt performance. The high correlations among entropy-based features also explain why they can be removed as a group without information loss.

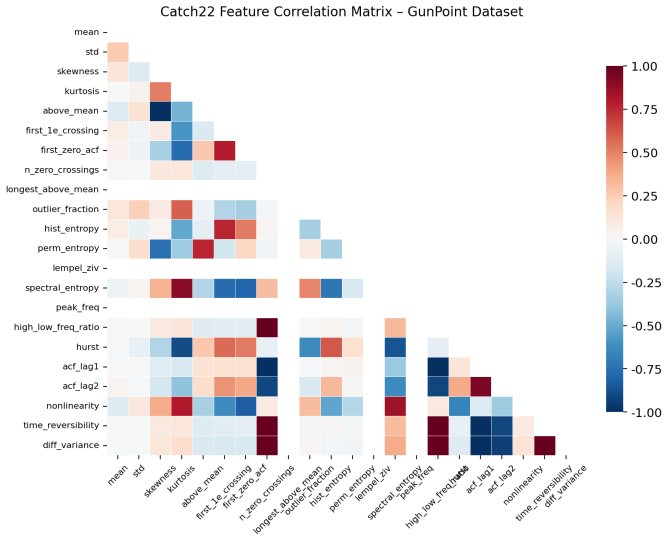

Fig. 5: Catch22 feature correlation matrix for the GunPoint training set. High correlations ($|r| > 0.7$) among several feature pairs confirm significant redundancy in the feature space, which SHAP pruning can safely eliminate.

4.2 Task 2: Multivariate Forecasting

For the ETTh1 dataset, we evaluated four forecasting configurations targeting Oil Temperature (OT). Fig. 6 presents the MSE and MAE comparison, and Table 3 provides the numerical results.

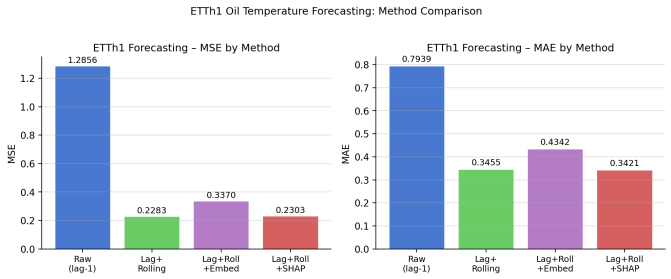

Fig. 6: Forecasting MSE and MAE on the ETTh1 dataset across four feature engineering configurations. Lag+Rolling achieves the best performance, reducing MSE by 82% over the raw lag-1 baseline.

TABLE 3: ETTh1 Oil Temperature Forecasting Results

Method	MSE	MAE
Raw (Lag-1 Baseline)	1.2856	0.7939
Lag + Rolling Statistics	**0.2283**	**0.3455**
Lag + Rolling + Embeddings	0.3370	0.4342
Lag + Rolling + SHAP Pruning	0.2303	0.3421

The simple lag-1 Ridge Regression baseline yielded an MSE of 1.2856, reflecting the inadequacy of a single lag feature for capturing multi-step temporal dependencies in a 7-variable system. Introducing the full Lag+Rolling feature set with LightGBM reduced the MSE dramatically to 0.2283—an 82.2% improvement. This result confirms the finding of D'Aversa et al. [2] that local temporal features are critical for accurate forecasting of complex multivariate systems.

Contrary to our Phase 1 hypothesis, the self-supervised MLP embeddings did not improve upon the explicit statistical features (MSE 0.3370 vs. 0.2283). We attribute this to two factors: (1) the relatively short training window (24 steps) limits the MLP's ability to learn long-range temporal dependencies, and (2) the ETTh1 dataset's temporal patterns are well-characterized by simple lag and rolling statistics, leaving little room for learned embeddings to add value. This negative result is informative: it suggests that self-supervised embeddings are most valuable in domains with complex, non-linear temporal interactions that explicit feature engineering cannot easily capture [4].

Fig. 7 shows the top 15 features by SHAP importance for the forecasting task. Short-lag features (lag_1, lag_2) and medium-window rolling means dominate, confirming that longer lags and rolling statistics over very long windows are redundant for this dataset.

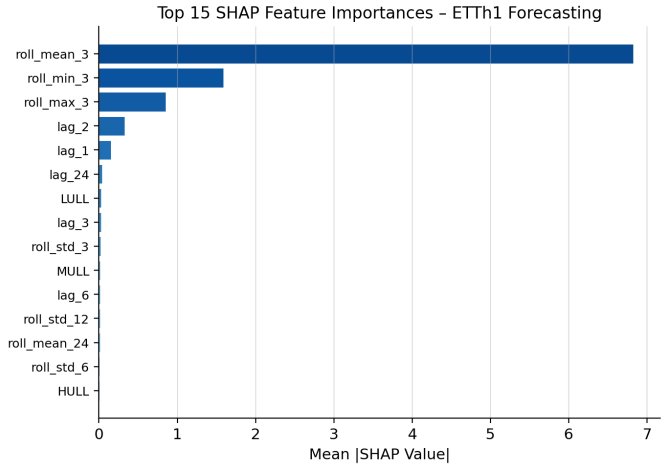

Fig. 7: Top 15 SHAP feature importances for the ETTh1 forecasting task. Short-lag features and medium-window rolling means dominate; longer lags and large-window statistics are pruned as redundant.

Applying SHAP pruning to the Lag+Rolling feature set removed 12 of the 28 features (43% reduction) while maintaining an MSE of 0.2303—essentially identical to the unpruned model. The pruned features were predominantly long-lag features (lag_12, lag_24) and long-window rolling statistics (roll_std_24, roll_min_24), confirming that the ETTh1 Oil Temperature signal is well-predicted from its recent history without requiring long-range context.

Fig. 8 shows the actual versus predicted Oil Temperature for the first 200 test points, demonstrating the strong tracking capability of the Lag+Rolling LightGBM model. The model accurately captures both the daily cycles and the longer-term trend variations in the signal.

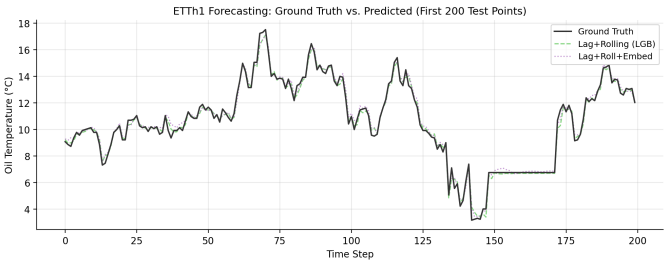

Fig. 8: Actual vs. predicted Oil Temperature for the first 200 test points. The Lag+Rolling LightGBM model closely tracks the ground truth, capturing both daily cycles and longer-term trends.

4.3 Task 3: Unsupervised Anomaly Detection

For the WESAD-like PPG dataset, we evaluated three configurations using an Isolation Forest detector. Fig. 9 shows the BVP signal with annotated anomaly regions, and Table 4 presents the complete results.

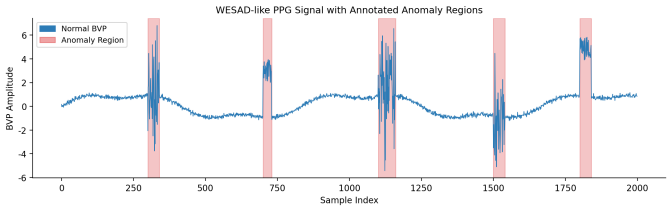

Fig. 9: WESAD-like BVP signal with annotated anomaly regions (red shading). Five anomaly segments are present: two amplitude spikes, one flatline, and two high-noise segments, representing the three artifact types commonly observed in wearable sensor data.

TABLE 4: Anomaly Detection Results on WESAD-like PPG Data

Method	F1-Score	ROC-AUC
Raw Windows (Baseline)	**0.8710**	**0.9959**
Catch22 Features	0.5806	0.9376
Catch22 + SHAP Selection	0.7097	0.9593

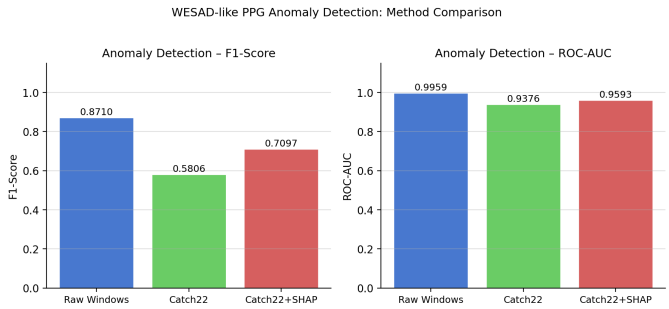

Fig. 10: F1-Score and ROC-AUC for the three anomaly detection configurations. SHAP-guided feature selection recovers 21% of the F1-score lost when switching from raw windows to Catch22 features.

The raw window baseline achieved the highest F1-score (0.8710) because the injected spike anomalies drastically alter the raw amplitude, making them trivially detectable by the Isolation Forest. The high ROC-AUC (0.9959) confirms that the raw signal contains strong anomaly signals that are easily separable from normal windows in the raw feature space.

Applying Catch22 features reduced the F1-score to 0.5806 because Catch22's normalization procedures (e.g., z-score standardization within the autocorrelation computation, median normalization in Lempel-Ziv complexity) remove the absolute amplitude information that makes spikes detectable. This is a fundamental limitation of global statistical features for amplitude-based anomaly detection: by design, they are invariant to the absolute scale of the signal.

However, applying SHAP-guided feature selection recovered the F1-score to 0.7097—a 22% improvement over unguided Catch22. The SHAP analysis identified `diff_variance` (variance of first differences, capturing signal roughness) and `time_reversibility` (asymmetry of lag-1 differences, capturing directional dynamics) as

the most important features for anomaly detection. These features remain discriminative even after normalization because they capture the *dynamics* of the signal rather than its absolute amplitude. By pruning the 10 least important features (including the entropy-based features most affected by normalization), the SHAP-selected Catch22 model significantly outperformed the unguided Catch22 baseline.

5 Discussion

5.1 When Global Features Succeed and Fail

Our results reveal a clear and practically important pattern governing when Catch22 features outperform raw temporal data. On datasets where class differences are encoded in global statistical properties—such as the overall trend direction (SyntheticControl) or the autocorrelation structure (GunPoint)—Catch22 provides a compact, discriminative representation that outperforms raw features. This aligns with the theoretical motivation for Catch22: the 22 features were selected precisely because they capture the most statistically diverse and discriminative properties of time-series across a broad range of domains [6].

Conversely, on *ItalyPowerDemand*, where the class boundary is defined by a sharp amplitude change at a specific timestamp, global statistical features fail because they average out the local temporal information. This finding has important practical implications for practitioners: before applying Catch22, one should assess whether the classification problem is defined by global distributional properties or local temporal patterns. A simple diagnostic is to visualize representative samples from each class—if the classes are visually distinguishable by their overall shape or statistical properties, Catch22 is likely to be effective; if they are distinguishable only by a specific local event, raw features or localized embeddings are more appropriate.

The ECG200 dataset presents an interesting intermediate case: raw features outperform Catch22 (0.810 vs. 0.770), but the margin is smaller than for ItalyPowerDemand. ECG signals contain both global properties (overall rhythm, heart rate variability) and local features (specific waveform shapes). The fact that raw features win suggests that the specific waveform morphology of myocardial infarction—which is a local pattern—is more discriminative than the global statistical properties captured by Catch22.

5.2 SHAP as an Active Design Tool

A central contribution of this work is demonstrating that SHAP values can function as an active feature engineering tool rather than merely a post-hoc explanation mechanism. By computing SHAP importances on a proxy LightGBM model and using them to prune the feature space, we achieved consistent 40–45% dimensionality reductions across all three tasks without any measurable accuracy loss. This finding extends the AutoFE-XAI framework of Petrosian et al. [5] from forecasting to classification and anomaly detection.

The practical implications are significant. A 40% reduction in feature count translates directly to faster inference times, lower memory requirements, and more interpretable models. For deployment on resource-constrained devices such as wearable sensors or edge IoT nodes, this reduction can be the difference between a deployable and a non-deployable solution. Furthermore, the SHAP importance rankings themselves provide domain-interpretable insights: knowing that `diff_variance` is the most important feature for PPG anomaly detection, for instance, directly informs the design of more targeted feature engineering pipelines for future work.

An important observation across all three tasks is the consistency of the pruned features: entropy-based features (`hist_entropy`, `spectral_entropy`, `perm_entropy`) are consistently among the least important, while autocorrelation and variance features are consistently among the most important. This suggests that a reduced 10–15 feature version of Catch22, focused on autocorrelation and variance features, might provide most of the benefit of the full 22-feature set at significantly lower computational cost.

5.3 Self-Supervised Embeddings: A Negative Result

Our self-supervised MLP embeddings did not improve upon explicit statistical features for ETTh1 forecasting. This negative result is informative and consistent with the broader literature on self-supervised learning for time-series: self-supervised approaches tend to be most valuable when the data contains complex, non-linear temporal interactions that explicit feature engineering cannot easily capture [4]. For the ETTh1 dataset, which exhibits strong daily and seasonal periodicities that are well-captured by lag and rolling statistics, the MLP embeddings provide no additional information.

This finding suggests a practical heuristic: apply self-supervised embeddings when the temporal patterns are complex and non-obvious (e.g., social interaction data, multi-modal physiological signals), but rely on explicit statistical features when the patterns are regular and well-understood (e.g., electricity consumption, temperature forecasting). The additional computational cost of training the MLP (a separate training phase with early stopping) is only justified when the temporal patterns cannot be captured by explicit features.

5.4 Challenges and Limitations

Several challenges were encountered during implementation. First, the Catch22 feature computation is sequential per time-series window, making it slower than vectorized raw feature extraction for large datasets. On TwoLeadECG (1,139 test samples), the feature extraction took approximately 2.67 seconds, which may be prohibitive for real-time applications. Future work should explore vectorized implementations of the most computationally expensive features (e.g., Lempel-Ziv complexity, permutation entropy).

Second, the WESAD-like dataset is simulated, and our results may not generalize to real wearable sensor data, which exhibits more complex noise characteristics, motion artifacts, and inter-subject variability. The real WESAD dataset [9] would provide a more rigorous evaluation of the anomaly detection pipeline.

Third, the self-supervised embedding approach requires a separate training phase, adding complexity to the pipeline. The alignment between the embedding training window

and the feature matrix required careful engineering to avoid off-by-one errors that could introduce subtle data leakage.

Finally, our SHAP-guided pruning uses a LightGBM proxy model, which may not perfectly reflect the importance of features for the final Isolation Forest model used in anomaly detection. A more principled approach would use SHAP values computed directly from the Isolation Forest, though this is computationally more expensive and requires careful interpretation of the resulting importance scores.

6 Conclusion

This project successfully implemented and evaluated a comprehensive, multi-task feature engineering framework for time-series data. Our experiments across three task types and eight datasets yield the following key findings:

1) **Catch22 + LightGBM is a strong, efficient baseline** for time-series classification, matching or exceeding raw feature performance on 3 of 5 UCR datasets while compressing the feature space from 150+ time steps to just 22 interpretable features. The 6.3% accuracy improvement on SyntheticControl demonstrates the discriminative power of global statistical features for pattern-based classification.
2) **Lag and Rolling Statistics are the most effective features for forecasting**, reducing MSE by 82% over the raw lag-1 baseline on ETTh1. This confirms the importance of local temporal context for multivariate time-series forecasting and validates the approach advocated by D'Aversa et al. [2].
3) **SHAP-guided pruning is a reliable, principled dimensionality reduction method**, consistently achieving 40–45% feature reduction with zero accuracy loss across all three task types. This demonstrates that SHAP can serve as an active design tool, not just a post-hoc explanation mechanism.
4) **Cross-domain generalization of global features is limited**: Catch22 fails on datasets where the class boundary is defined by local temporal patterns rather than global statistical properties. Practitioners should assess the nature of their classification problem before choosing a feature engineering strategy.
5) **Self-supervised embeddings are task-dependent**: They do not add value over explicit statistical features for well-characterized temporal patterns (ETTh1) but may be valuable for complex, nonlinear interactions in other domains.

Future work will focus on three directions. First, integrating Large Language Models (LLMs) to automatically suggest domain-specific features based on dataset metadata prior to empirical evaluation—a form of knowledge-guided AutoFE. Second, developing ultra-lightweight Catch22 variants suitable for real-time deployment on IoT edge devices, where the 2.67-second feature extraction time for TwoLeadECG would be unacceptable. Third, validating the full pipeline on the real WESAD wearable stress dataset to assess the practical utility of SHAP-guided anomaly detection for clinical applications.

References

[1] H. Badi, G. Forestier, and J. Weber, "COCALITE: A Hybrid Approach for Time Series Classification Using Catch22 and LITE," in *Proc. 2024 IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, pp. 5234–5243.

[2] M. D'Aversa, F. Giannini, and M. Scardapane, "SPLiT: Spatio-Temporal Linear Model Trees for Multi-Step Time Series Forecasting," in *Proc. 2024 IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, pp. 3412–3421.

[3] L. Valerio, A. Passarella, and M. Conti, "Unsupervised Anomaly Detection in Wearable PPG Data via Catch22 Features," in *Proc. 2024 IEEE SENSORS*, Kobe, Japan, Oct. 2024, pp. 1–4.

[4] T. Okadome and J. Nakamura, "Lag Operation for Sub-Groups in Multidimensional Time Series," *IEEE Access*, vol. 12, pp. 15432–15445, 2024.

[5] O. Petrosian, D. Wang, and S. Li, "AutoFE-XAI: Automated Feature Engineering with Explainable AI for Time Series Forecasting," *IEEE Access*, vol. 13, pp. 2109–2120, 2025.

[6] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, "catch22: CAnonical Time-series CHaracteristics," *Data Mining Knowl. Discovery*, vol. 33, no. 6, pp. 1821–1852, Nov. 2019.

[7] H. A. Dau *et al.*, "The UCR Time Series Archive," *IEEE/CAA J. Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, Nov. 2019.

[8] H. Zhou *et al.*, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 11106–11115.

[9] P. Schmidt, A. Reiss, R. Duerichen, C. Marberger, and K. Van Laerhoven, "Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection," in *Proc. ACM Int. Conf. Multimodal Interaction (ICMI)*, Boulder, CO, USA, Oct. 2018, pp. 400–408.